Faster convex coverage sets via metapolicies over base policies

Guillem Masdemont, Vicenç Gómez

October 26, 2026

Abstract

In Multi-Objective Reinforcement Learning, the Convex Coverage Set (CCS) combined with Generalized Policy Improvement (GPI) enables optimal transfer from a set of learnt tasks to unseen ones. However, reaching optimality often requires computing a large number of policies. This becomes infeasible in large MDPs, as the size of the CCS grows exponentially with the number of objectives. In this work, we present a metapolicy that chooses which base policy's action to take at every state, so that new policies can be composed from a small basis. The metapolicy is the optimal policy of the MDP restricted to the actions of the base policies, and its successor features are learned with a restricted form of SF Q-learning. This restriction trades the expressiveness of other approaches, such as the Option Keyboard, for efficient learning. We prove the convergence of our method and show that it reduces the number of samples needed to construct the CCS with respect to SFOLS in deterministic domains, while under stochastic transitions the gain is smaller, up to 23% fewer samples than the warm-started SFOLS, since learning the successor features of the basis dominates the cost. Furthermore, we propose a way to combine the metapolicy with GPI into a hybrid policy whose action values are provably at least those of GPI over the same basis, enabling better performance on unseen tasks when computing the whole CCS is still infeasible. Finally, we use these tools in SF-FSA-VI to solve non-Markovian tasks: the extreme policies are enough in deterministic domains, and in stochastic domains Lazy SF-FSA-VI only adds policies at the weights that a task needs. With exact successor features it is optimal on every task with about two new policies per task, and without a model it matches SF-FSA-VI over SFOLS with 2–4× fewer samples, while most new tasks need no new policy. These findings open new directions for optimal policy transfer in large domains, where only a small set of base policies can be computed in a reasonable amount of time.

1 Introduction

Multi-objective reinforcement learning (MORL) models Markov decision processes whose reward function signals several, often conflicting, objectives [1, 2]. The agent must act optimally under any possible preference weighting, hence, the relevant solution is the whole Pareto frontier of optimal value vectors, which is expensive to compute because it grows exponentially with the number of objectives. This motivates the *optimal policy transfer* problem: achieving optimality on new tasks by combining previously learnt policies [3].

In the linear case where the reward is a scalar product of objectives and weights, one can efficiently compute the value function of a policy across different tasks using successor features (SFs) [4, 5]. Moreover, the optimal Pareto frontier forms a convex hull that can be leveraged using linear programming methods. While the learnt policies can be reused to improve performance on a new task, optimality depends on how diverse the basis set is. Zahavy et al. [6] construct this set iteratively by adding the policy that most improves the worst-case performance of the composed, set-max policy over all tasks. Alternatively, for deterministic dynamics and independent reward features, Alver and Precup [7] show that a set of independent policies, each maximizing its own feature without affecting the others, allows to maximize the expected return on every linearly task. Nevertheless, global optimallity is only guaranteed when the base policies forms a *convex coverage set* (CCS), a minimal set of policies that contains an optimal policy for every preference [4]. Optimistic linear support (OLS) builds the CCS incrementally by solving the task at the corner weights of the current set [4], and SFOLS [3] combines it with SFs and *General policy improvement* (GPI) to achieve optimality on every linear task.

However, the number of policies that must be learned from scratch grows rapidly with the number of objectives, which makes SFOLS unfeasible in many domains. Recent works trade this guarantee for scalability: constructing a smaller behavior basis that is combined by a learned meta-policy, the option keyboard [8], relying on independent policies [7], or discovering the features and approximating the SFs [9]. When the whole Pareto frontier is not even desirable one can restrict it with Lorenz dominance to favour specific reward distributions [10].

In this work we reduce the cost of building the CCS by reusing previously learnt policies. Instead of learning a new policy from scratch, we try to compose it from a basis of previous policies through a *metapolicy* that, at every state, chooses which base policy's action to take, and refines this choice

by policy iteration. A Bellman certificate then checks whether the composed policy is already optimal for the queried weight. If it is not, then we learn the new policy and add it to the basis. In particular, this work demonstrates that:

- Our Certify-or-Grow method recovers the same CCS as SFOLS with 3.2× fewer environment steps until convergence than the warm-started SFOLS in the Deep Sea Treasure domain and 2.1× fewer in the Office domain, averaged over 10 seeds, also when the certificate is checked without a model. Under stochastic transitions it needs up to 23% fewer steps until convergence and stops up to 2.4× earlier, although learning the successor features of the basis dominates the cost.

- The metapolicy can be extended with GPI and is provably at least as good as the GPI estimate over the same basis. In particular, we show in the stochastic double-slit domain it can be leveraged to greatly improve performance on a new task as a replacement of increasing the CCS.

- In SF-FSA-VI, the extreme policies are enough in deterministic domains, and in stochastic domains Lazy SF-FSA-VI is optimal on every task with about 2 added policies per task, instead of an ε-CCS of 117–572 policies. When the added policies are kept, most new tasks need none, also without a model, where it matches SF-FSA-VI over SFOLS with 2–4× fewer samples.

We present results on both deterministic and stochastic environments, covering both Markovian and non-Markovian tasks. The Deep Sea Treasure domain is taken from the MORL benchmarking toolkit [11], while the Office and double slit domains come from Kuric et al. [12]. In the non-Markovian setting, learned SFs are composed over a finite state automaton [12], in the spirit of options [13] and reward machines [14]. The remainder of the work is structured as follows. In Sec. 2 we introduce the notation, SFs, GPI and the CCS. In Sec. 3 we define the metapolicy, the Bellman certificate and Certify-or-Grow, and show that the hybrid policy improves on GPI. In Sec. 4 we use them in SF-FSA-VI: the extreme policies are enough in deterministic domains, and in stochastic ones Lazy SF-FSA-VI only adds policies at the weights that a task needs. In Sec. 5 we evaluate our methods in the Deep Sea Treasure, Office, double slit and windy domains. We conclude in Sec. 6.

2 Background and notation

An MDP is a tuple $\mathcal{M} = (S, A, P, r, \gamma)$, where S is the state space, A is the action space, $P(\cdot|s, a)$ is the transition distribution from taking action a at state s, $r : S \times A \times S \to \mathbb{R}$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. In the context of MORL, we consider a vector reward function with d objectives represented by a feature vector $\boldsymbol{\phi}(s, a, s') \in \mathbb{R}^d$. Every MORL problem induces a family of MDPs via a scalarization function that maps the vector of objectives to a scalar reward [1]. In the linear case, every weight $\mathbf{w}$ induces a task, given by the MDP $\mathcal{M}_{\mathbf{w}} = (S, A, P, r_{\mathbf{w}}, \gamma)$ with reward $r_{\mathbf{w}}(s, a, s') = \boldsymbol{\phi}(s, a, s') \cdot \mathbf{w}$. We denote by $V^*_{\mathbf{w}}$ the optimal value function of $\mathcal{M}_{\mathbf{w}}$, and by $\pi^*_{\mathbf{w}}$ an optimal policy.

Without loss of generality, we can assume that $\mathbf{w}$ is a probability distribution, i.e., $\mathbf{w} \in \Delta(d)$, where $\Delta(d) = \{\mathbf{w} \in \mathbb{R}^d \mid \sum_{i=1}^d w_i = 1, w_i \geq 0\}$. Negative weights are also considered in other works [5], however, in that case the optimal policies are no longer restricted to the Pareto frontier, and the methods that rely on its convexity do not apply directly [15, 10]. In this work the features $\boldsymbol{\phi}$ are given, although in more general settings they can also be learned from experience together with weights [5, 9].

Given a policy π, we define its successor features (SFs) [16] by

$$\boldsymbol{\psi}^{\pi}(s, a) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{\phi}(s_t, a_t, s_{t+1}) \,\middle|\, s_0 = s, a_0 = a\right]. \tag{1}$$

The linearity in the reward allows $q^{\pi}_{\mathbf{w}}(s, a) = \boldsymbol{\psi}^{\pi}(s, a) \cdot \mathbf{w}$. Moreover, the SFs satisfy the Bellman equation $\boldsymbol{\psi}^{\pi}(s, a) = \mathbb{E}_{s'}\left[\boldsymbol{\phi}(s, a, s') + \gamma \boldsymbol{\psi}^{\pi}(s', \pi(s'))\right]$, so they can be computed with policy evaluation or learned with temporal-difference methods.

Given a set of policies $\Pi = \{\pi_i\}_{i=1}^N$ and their corresponding successor features $\{\boldsymbol{\psi}^{\pi_i}(s, a)\}$, generalized policy improvement (GPI) [16] defines a new policy via

$$\pi^{GPI}(s) = \arg\max_{a \in A} \max_{\pi \in \Pi} \boldsymbol{\psi}^{\pi}(s, a) \cdot \mathbf{w}. \tag{2}$$

For any weight $\mathbf{w}$, π^{GPI} is proven to perform at least as well as all the policies in Π, because $q^{\pi^{GPI}}_{\mathbf{w}}(s, a) \geq \max_{\pi \in \Pi} q^{\pi}_{\mathbf{w}}(s, a)$ for every $(s, a) \in S \times A$. Note that GPI is a one-step improvement: it evaluates every action assuming that a single policy of Π is followed afterwards. In general it is not optimal and it can perform poorly when the optimal behaviour requires switching between policies [17]. However, GPI is guaranteed to be optimal if Π covers all the possible optimal behaviours, i.e., if for any weight $\mathbf{w}$ there

exists $\pi \in \Pi$ optimal for $\mathcal{M}_{\mathbf{w}}$. Given an initial state distribution ν_0, let $\boldsymbol{\psi}^{\pi} = \mathbb{E}_{s\sim\nu_0}\left[\boldsymbol{\psi}^{\pi}(s, \pi(s))\right]$ be the expected return vector of π. The optimal policies from ν_0 lie in the Pareto frontier given by the dominance relation $\boldsymbol{\psi}^{\pi} \preceq \boldsymbol{\psi}^{\pi'} \iff \psi_i^{\pi} \leq \psi_i^{\pi'}$ for all $i \in \{1, \ldots, d\}$. For the linear case it is enough to keep its convex part, the *convex coverage set* (CCS) [4, 3], which contains the return vectors that are optimal for some task:

$$CCS = \left\{ \boldsymbol{\psi}^{\pi} \,\middle|\, \exists \mathbf{w} \in \Delta(d) \text{ such that } \boldsymbol{\psi}^{\pi} \cdot \mathbf{w} \geq \boldsymbol{\psi}^{\pi'} \cdot \mathbf{w} \;\; \forall \pi' \right\}. \tag{3}$$

If Π contains the policies that define the CCS, then π^{GPI} is optimal for every weight [3]. Note that the CCS is defined with respect to ν_0, so it is important to have a good distribution of initial states if one wants optimality from every initial condition, otherwise the CCS may miss optimal policies for unexplored initial states.

Given a partial CCS, $\max_{\pi\in\Pi} \boldsymbol{\psi}^{\pi} \cdot \mathbf{w}$ is a piecewise-linear convex function of $\mathbf{w}$, and its *corner weights* are the weights where the linear pieces intersect, namely, the vertices of the polytope $\{(\mathbf{w}, v) \,|\, \mathbf{w} \in \Delta(d),\ v \geq \boldsymbol{\psi}^{\pi} \cdot \mathbf{w}\ \forall \pi \in \Pi\}$.

Theorem 1 ([4])**.** *Given a set of policies* Π*, the maximum gap between the optimal value and the value of* Π*,*

$$\max_{\mathbf{w}\in\Delta(d)} \left(V^*_{\mathbf{w}} - \max_{\pi\in\Pi} \boldsymbol{\psi}^{\pi} \cdot \mathbf{w} \right), \tag{4}$$

is always attained at a corner weight of Π*.*

Theorem 1 ensures that if the optimal policy of every corner weight is already in Π, the set is a CCS. Optimistic linear support (OLS) [4] uses this to build the CCS incrementally by starting in the extreme weights and querying corner weights until no improvement. SFOLS (Algorithm 1) combines OLS with SFs and GPI [3]. Each queried weight is solved starting from the GPI policy over the policies learned so far, and the corner weights are prioritized by an upper bound on their gap, so that the most informative one is explored first. The order of the queries does not affect convergence, but it can improve its speed, since a weight close to an already explored one may give the same optimal policy, and hence the same SFs. Other prioritizations are proposed in [18].

Algorithm 1 SFOLS [3]

$\Pi \leftarrow \emptyset$, $CCS \leftarrow \emptyset$, $Q \leftarrow \{\mathbf{e}_1, \ldots, \mathbf{e}_d\}$ ▷ queue of weights to query
while $Q \neq \emptyset$ **do**
  $\mathbf{w} \leftarrow$ pop weight of Q with highest priority
  $\pi_{\mathbf{w}}, \boldsymbol{\psi}^{\pi_{\mathbf{w}}} \leftarrow$ solve $\mathcal{M}_{\mathbf{w}}$
  $\Pi \leftarrow \Pi \cup \{\pi_{\mathbf{w}}\}$
  if $\boldsymbol{\psi}^{\pi_{\mathbf{w}}} \notin CCS$ and it improves $\max_{\pi \in \Pi} \boldsymbol{\psi}^{\pi} \cdot \mathbf{w}$ **then**
    $CCS \leftarrow CCS \cup \{\boldsymbol{\psi}^{\pi_{\mathbf{w}}}\}$, and remove the corner weights of Q that are no longer corners
    add the new corner weights to Q, with priority given by an upper bound on their gap
  end if
end while
return CCS, Π

SFOLS often performs redundant solves. For example, the Deep Sea Treasure environment [19] needs 19 weight solves, of which only 10 yield different policies. Moreover, the size of the CCS grows exponentially with the number of objectives, so in many cases computing it is very expensive and not even necessary. For example, in the Office domain [12], the policies of the d extreme weights are already enough to achieve zero-shot optimality with GPI. The reason is that, in most problems, the optimal policy for a new weight is the behaviour of a few base policies over broad regions of the state space, and only switches between them at specific states. Hence, a new policy often does not need to be learned from scratch, as long as we can decide when the cached policies are enough. Nemecek and Parr [20] address this decision with bounds on the optimal value of a new task, which tell whether to reuse the cached policies with GPI or to learn a new one.

Recent works reduce the number of policies in the basis, in exchange for a richer way of combining them. Some works build the basis from the eigenvectors of the graph Laplacian of the environment and combine it with GPI [21], while others learn a successor feature approximator jointly with the features and the weights [22, 9]. The Option Keyboard [23] combines a basis of options through a learned metapolicy that outputs, at each state, the weight used to compose them with GPI, and Alegre et al. [8] recently showed how to construct a basis for it that is optimal for every task.

$$\pi^{OK}(s) = \arg\max_{a \in A} \max_{\pi \in \Pi} \boldsymbol{\psi}^{\pi}(s, a) \cdot \boldsymbol{\omega}(s, \mathbf{w}), \tag{5}$$

where $\boldsymbol{\omega} : S \times \Delta(d) \to \mathbb{R}^d$ is a trained metapolicy. Changing the weight

at each state allows π^{OK} to produce behaviours that no base policy has. However, learning $\boldsymbol{\omega}$ is usually hard since the optimal policy is mostly a composition of base policies over broad regions and the target $\boldsymbol{\omega}$ becomes a step function. Moreover, the arg max in Eq. (5) partitions the weight space into a mesh of open cells, each of them selecting an action. Hence, small enough changes on $\boldsymbol{\omega}$ do not change the action output and the gradient vanishes. Finally, Eq. (5) does not restrict the actions to those of the base policies, so for ω to reach the cell of the optimal action, the learning process may have to move through cells whose actions are not optimal.

In this paper, we exploit these observations and present a metapolicy for MDPs that only allows, at each state, the actions of the base policies. Since each base policy π_j is optimal for its training weight $\mathbf{w}_j$, taking $\boldsymbol{\omega}(s, \mathbf{w}) = \mathbf{w}_j$ in Eq. (5) recovers exactly the action $\pi_j(s)$, so this is equivalent to restricting the option keyboard to discrete choices. This discrete choice can be learned efficiently, and it proves to be faster than SFOLS in the Deep Sea Treasure and Office domains.

3 Formalism

We first describe how to compute a metapolicy from a set of base policies, then detail the construction of the CCS. Throughout this section, the query weight $\mathbf{w}$ remains fixed.

3.1 Metapolicy

We call *basis* a finite set of policies $\mathcal{B} = \{\pi_j\}_{j=1}^{k}$, together with their successor features $\{\boldsymbol{\psi}^{\pi_j}(s, a)\}$. Each π_j is optimal for its own training weight $\mathbf{w}_j$, hence, $\pi_j(s) = \arg\max_{a \in A} \boldsymbol{\psi}^{\pi_j}(s, a) \cdot \mathbf{w}_j$.

For any state s, consider the restricted action set given by the actions of the base policies:

$$B(s) = \{\pi_1(s), \ldots, \pi_k(s)\} \subseteq A. \tag{6}$$

and define $\mathcal{M}_{\mathcal{B}}$ as the MDP with the same states, transitions and features as $\mathcal{M}$, but with action set $B(s)$ at every state. In $\mathcal{M}_{\mathcal{B}}$ the agent chooses which base policy to follow at each state. Since the optimal policy is often a composition of the base policies over broad regions, it is frequently contained in $\mathcal{M}_{\mathcal{B}}$.

We define the *metapolicy* π_{meta} over $\mathcal{B}$ as the optimal policy of $\mathcal{M}_{\mathcal{B}}$ for the weight $\mathbf{w}$, and V_{meta} as its value. For any value function V, we write

the one-step backup as

$$Q_V(s,a) = \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[\boldsymbol{\phi}(s,a,s') \cdot \mathbf{w} + \gamma V(s') \right], \tag{7}$$

The Bellman optimality operator of $\mathcal{M}_{\mathbf{w}}$ is $(TV)(s) = \max_{a \in A} Q_V(s,a)$, with unique fixed point $V^*_{\mathbf{w}}$. Equivalently, V_{meta} is the unique fixed point of the restricted Bellman operator

$$(T_B V)(s) := \max_{a \in B(s)} Q_V(s,a) = \max_{j \in \{1,\dots,k\}} Q_V(s, \pi_j(s)), \tag{8}$$

which is also a γ-contraction, and the metapolicy follows, at each state, the best base policy:

$$\pi_{\text{meta}}(s) = \pi_{j^*(s)}(s), \qquad j^*(s) = \arg \max_{j \in \{1,\dots,k\}} Q_{V_{\text{meta}}}(s, \pi_j(s)). \tag{9}$$

Note that the metapolicy can never pick an action that none of the base policies would take. Moreover, since every base policy is itself a policy of $\mathcal{M}_{\mathcal{B}}$, the metapolicy is at least as good as all of them.

Lemma 1 (V_{meta} dominates the base policies)**.** *For every base policy $\pi_j \in \mathcal{B}$ and every state s, $V_{\text{meta}}(s) \geq V^{\pi_j}(s)$, where $V^{\pi_j}(s) = \boldsymbol{\psi}^{\pi_j}(s, \pi_j(s)) \cdot \mathbf{w}$.*

Proof. Since $\pi_j(s) \in B(s)$, the Bellman equation of π_j gives $V^{\pi_j}(s) = Q_{V^{\pi_j}}(s, \pi_j(s)) \leq (T_B V^{\pi_j})(s)$. Applying the monotone operator T_B repeatedly gives a non-decreasing sequence $V^{\pi_j} \leq T_B V^{\pi_j} \leq T_B^2 V^{\pi_j} \leq \dots$, which converges to V_{meta} by the contraction property. □

3.2 Learning the metapolicy and its successor features

In practice, we focus on learning the SFs of π^{meta} rather than its value function as SFs are required to perform GPI. Hence, instead of iterating T_B and then evaluating the resulting policy, we learn both at once with SF Q-learning [5], where the bootstrap is restricted to the actions of the base policies (Algorithm 2).

Algorithm 2 MetaPolicy($\mathcal{B}, \mathbf{w}$)

$\widehat{\boldsymbol{\psi}}(s,a) \leftarrow \boldsymbol{\psi}^{\pi_j}(s,a)$ with $j = \arg\max_j \boldsymbol{\psi}^{\pi_j}(s,a) \cdot \mathbf{w}$, for every (s,a)
repeat
  observe $(s, a, \boldsymbol{\phi}(s,a,s'), s')$ under a behaviour policy with enough state-action coverage
  $\hat{\jmath} \leftarrow \arg\max_{j \in \{1,\dots,k\}} \widehat{\boldsymbol{\psi}}\big(s', \pi_j(s')\big) \cdot \mathbf{w}$
  $\widehat{\boldsymbol{\psi}}(s,a) \leftarrow \widehat{\boldsymbol{\psi}}(s,a) + \alpha \left[\boldsymbol{\phi}(s,a,s') + \gamma\, \widehat{\boldsymbol{\psi}}\big(s', \pi_{\hat{\jmath}}(s')\big) - \widehat{\boldsymbol{\psi}}(s,a)\right]$
until converged
$\pi_{\text{meta}}(s) \leftarrow \pi_{j^*(s)}(s)$, with $j^*(s) = \arg\max_j \widehat{\boldsymbol{\psi}}(s, \pi_j(s)) \cdot \mathbf{w}$, for every s
return π_{meta}, $\widehat{\boldsymbol{\psi}}$

The update has two parts: The arg max of the target only ranges over $B(s')$, therefore, the scalar values $\widehat{\boldsymbol{\psi}}(s,a) \cdot \mathbf{w}$ follow exactly the Q-learning update of $\mathcal{M}_{\mathcal{B}}$. Hence they converge to V_{meta}, and the greedy choice $\hat{\jmath}$ converges to the metapolicy of Eq. (9). Second, the scalar values also converge for every pair (s,a) visited by the behaviour policy, including the actions $a \notin B(s)$, and if the best action of $\mathcal{M}_{\mathcal{B}}$ is unique at every state, $\widehat{\boldsymbol{\psi}}$ converges to $\boldsymbol{\psi}^{\pi_{\text{meta}}}(s,a)$. We prove both results, and discuss the case of ties, in Appendix A. Algorithm 2 can be speeded-up by improving sample efficiency, as described in [18]. The table can also be warm started with GPI over the basis together with all the metapolicies certified so far.

3.3 Bellman certificate

The metapolicy is optimal on $\mathcal{M}_{\mathcal{B}}$, but not necessarily on $\mathcal{M}$, since the optimal policy may need an action that is not in $B(s)$. We define the optimality gap as $\text{gap}_{\mathbf{w}}(s) = (TV_{\text{meta}})(s) - V_{\text{meta}}(s)$. Here T is the unrestricted Bellman operator. This gap can be read directly from the table returned by Algorithm 2 as

$$\text{gap}_{\mathbf{w}}(s) = \max_{a \in A} \boldsymbol{\psi}^{\pi_{\text{meta}}}(s,a) \cdot \mathbf{w} - \boldsymbol{\psi}^{\pi_{\text{meta}}}(s, \pi_{\text{meta}}(s)) \cdot \mathbf{w}. \tag{10}$$

Note that $\text{gap}_{\mathbf{w}}(s) \geq 0$ for every state, because $B(s) \subseteq A$. If the gap is zero at every state, then V_{meta} is also a fixed point of T, hence $V_{\text{meta}} = V^*_{\mathbf{w}}$ and the metapolicy is optimal for $\mathbf{w}$. More generally, if $\max_s \text{gap}_{\mathbf{w}}(s) \leq \varepsilon$, the contraction of T gives

$$\begin{aligned} \|V^*_{\mathbf{w}} - V_{\text{meta}}\|_\infty &\leq \|TV^*_{\mathbf{w}} - TV_{\text{meta}}\|_\infty + \|TV_{\text{meta}} - V_{\text{meta}}\|_\infty \\ &\leq \gamma \|V^*_{\mathbf{w}} - V_{\text{meta}}\|_\infty + \varepsilon, \end{aligned} \tag{11}$$

so the metapolicy is $\varepsilon/(1-\gamma)$-optimal.

Note that Eq. (10) needs $\boldsymbol{\psi}^{\pi_{\text{meta}}}(s,a)$ also for the actions outside $B(s)$, which Algorithm 2 does not visit. Once π_{meta} is learned, each of them follows from a single backup, $\boldsymbol{\psi}^{\pi_{\text{meta}}}(s,a) = \mathbb{E}_{s'}\left[\boldsymbol{\phi}(s,a,s') + \gamma \boldsymbol{\psi}^{\pi_{\text{meta}}}(s', \pi_{\text{meta}}(s'))\right]$, since the right-hand side only uses entries of the actions in $B(s')$, which are already learned. With a known model this backup is computed directly. Without it, we take each missing action once and use the observed transition, which is exact in deterministic domains; in stochastic domains a few samples per pair have to be averaged. These extra samples are counted in our results.

3.4 Certify-or-Grow

We now present CERTIFY-OR-GROW (Algorithm 3), which uses the certificate to answer a query weight. It first learns the metapolicy over the current basis. If the certificate holds, the metapolicy is returned without learning any new policy. Otherwise, we solve the task at $\mathbf{w}$ over the full action set, and the new policy is added to the basis, so that the next queries can reuse it.

Algorithm 3 CERTIFY-OR-GROW$(\mathbf{w}, \mathcal{B})$

1: $\pi_{\text{meta}},\ \boldsymbol{\psi}^{\pi_{\text{meta}}} \leftarrow$ METAPOLICY$(\mathcal{B}, \mathbf{w})$ ▷ Algorithm 2
2: **if** $\max_s \text{gap}_{\mathbf{w}}(s) \leq \varepsilon$ **then** ▷ Eq. (10)
3: **return** $\pi_{\text{meta}},\ \boldsymbol{\psi}^{\pi_{\text{meta}}},\ \mathcal{B}$
4: **else**
5: $\pi_{\mathbf{w}},\ \boldsymbol{\psi}^{\pi_{\mathbf{w}}} \leftarrow$ solve $\mathcal{M}_{\mathbf{w}}$ over the full action set
6: $\mathcal{B} \leftarrow \mathcal{B} \cup \{\pi_{\mathbf{w}}\}$
7: **return** $\pi_{\mathbf{w}},\ \boldsymbol{\psi}^{\pi_{\mathbf{w}}},\ \mathcal{B}$
8: **end if**

To build the CCS, the OLS loop of SFOLS (Algorithm 1) remains unchanged, and we only replace the solve at each corner weight (line 4) by Algorithm 3. Since a certified metapolicy is optimal for its weight, the CCS obtained is the same as with SFOLS, but many of its elements are composed instead of learned from scratch.

3.5 Combining the metapolicy with GPI

The metapolicy might not beat GPI over the same basis because GPI can also pick actions that no base policy would ever take. We now define a policy that combines both. We call *instant GPI value* the quantity

that GPI maximizes, $Q_{\mathbf{w}}^{GPI}(s,a) = \max_j \boldsymbol{\psi}^{\pi_j}(s,a) \cdot \mathbf{w}$, so that $\pi^{GPI}(s) = \arg\max_a Q_{\mathbf{w}}^{GPI}(s,a)$. For every j and action $a \in A$, the immediate reward of $Q_{\mathbf{w}}^{\pi_j}(s,a)$ does not depend on j, hence

$$Q_{\mathbf{w}}^{GPI}(s,a) = \mathbb{E}_{s'}\left[\boldsymbol{\phi}(s,a,s') \cdot \mathbf{w}\right] + \gamma \max_j \mathbb{E}_{s'}\left[V^{\pi_j}(s')\right]. \tag{12}$$

That is, GPI evaluates every action by following a single base policy afterwards. By Lemma 1, substituting V^{π_j} by V_{meta} in Eq. (12) can only increase it, and by the Bellman equation of π_{meta} the result is its action value. Hence

$$Q_{\mathbf{w}}^{GPI}(s,a) \leq Q_{\mathbf{w}}^{\text{meta}}(s,a) = \boldsymbol{\psi}^{\pi_{\text{meta}}}(s,a) \cdot \mathbf{w} \quad \forall (s,a). \tag{13}$$

This motivates the *hybrid policy*, which can take any action of A but scores it with the value of the metapolicy:

$$\pi^{\text{hyb}}(s) = \arg\max_{a \in A} \boldsymbol{\psi}^{\pi_{\text{meta}}}(s,a) \cdot \mathbf{w}. \tag{14}$$

Since π^{hyb} is greedy with respect to $Q_{\mathbf{w}}^{\text{meta}}$, the policy improvement theorem gives $Q_{\mathbf{w}}^{\pi^{\text{hyb}}} \geq Q_{\mathbf{w}}^{\text{meta}} \geq Q_{\mathbf{w}}^{GPI}$, namely, the hybrid policy always achieves at least the instant GPI value. However, this does not guarantee that π^{hyb} beats the policy π^{GPI} when it is executed. Both policies score each action assuming a fixed continuation, the metapolicy or a single base policy, but when executed they choose again at every state over the full action set, and their actual continuation can be better than the one they assumed. Hence, π^{GPI} can occasionally reach states where such an improvement is larger, and obtain more than π^{hyb}. Nevertheless, this is rare in practice: over the tested weights, π^{hyb} never performed worse than π^{GPI} in the Deep Sea Treasure and Office domains Moreover, π^{hyb} is just the greedy policy of the table $\boldsymbol{\psi}^{\pi_{\text{meta}}}$ over the full action set, which CERTIFY-OR-GROW already computes, so it comes for free on top of our algorithm.

Hence, the hybrid policy can be used in two ways. When the certificate fails, it can help warm start the solve of line 5 in Algorithm 3, and in the same way that [6] uses the value of the set-max policy v^{SMP}, it can be used as a heuristic to choose which weights to explore. Moreover, when building the whole CCS is not feasible, π^{hyb} can be used directly on a new task as a replacement for growing the basis, as we show in the double slit domain in Sec. 5.

4 SF-FSA-VI algorithm

We now use the previous sections to build a complete algorithm for SF-FSA-VI [12]. In this setting, the low-level MDP has d exit states $\mathcal{E} = \{\varepsilon_1, \dots, \varepsilon_d\}$, and the feature vector is $\boldsymbol{\phi}(s, a, s') = \mathbf{e}_j$ when $s' = \varepsilon_j$ and $\mathbf{0}$ otherwise. The task is a finite state automaton with states $\mathcal{U}$, where reaching the exit ε_j in state u moves the automaton to $\tau(u, j)$. SF-FSA-VI computes a weight vector $\mathbf{w}(u)$ for every $u \in \mathcal{U}$ by iterating

$$w_j^{k+1}(u) = \max_{a \in A} \max_{\pi \in \Pi} \boldsymbol{\psi}^{\pi}(\varepsilon_j, a) \cdot \mathbf{w}^k\big(\tau(u, j)\big), \tag{15}$$

with $w_j(u) = 1$ when $\tau(u, j)$ is the accepting state, and then acts with GPI at $\mathbf{w}(u)$ in every automaton state. Its main bottleneck is that Eq. (15) is only exact when Π is a CCS. In this section we show that, when the low-level MDP is deterministic, the d policies of the extreme weights $\mathbf{e}_1, \dots, \mathbf{e}_d$ are enough to achieve optimality for any weight. Under stochastic transitions they are not, and we leverage the metapolicy to improve the algorithm.

4.1 SF-FSA-VI for deterministic domains

Theorem 2. *Let the low-level MDP be deterministic, with the exit features above, and let π_k be an optimal policy for $\mathbf{e}_k$, for $k = 1, \dots, d$. Then, for every (s, a) and every weight $\mathbf{w} \in \Delta(d)$:*

$$\max_k \boldsymbol{\psi}^{\pi_k}(s, a) \cdot \mathbf{w} = Q^*_{\mathbf{w}}(s, a). \tag{16}$$

As a consequence, GPI over the d extreme policies is optimal for every weight, and SF-FSA-VI over them is optimal for every task.

Proof. Under the hypothesis, $Q^*_{\mathbf{w}}(s, a)$ ends in at most one exit, say ε_k after t steps, hemce, returns $w_k \gamma^t$. If the sequence never ends returns 0 which is worse. Moreover, the best way to reach ε_k is the fastest one, whose discounted value is $\psi_k^{\pi_k}(s, a)$. Hence $Q^*_{\mathbf{w}}(s, a) = \max_k w_k \psi_k^{\pi_k}(s, a)$. On the other hand, since the features and the weights are non-negative, $\boldsymbol{\psi}^{\pi_k}(s, a) \cdot \mathbf{w} \geq w_k \psi_k^{\pi_k}(s, a)$, so $\max_k \boldsymbol{\psi}^{\pi_k}(s, a) \cdot \mathbf{w} \geq Q^*_{\mathbf{w}}(s, a)$. Finally, $\boldsymbol{\psi}^{\pi_k}(s, a) \cdot \mathbf{w} = Q^{\pi_k}_{\mathbf{w}}(s, a) \leq Q^*_{\mathbf{w}}(s, a)$, and the sandwich argument gives the equality. □

Moreover, since $\pi_k(s)$ is the first step of a fastest path to ε_k, the optimal action at every state is the action of one of the extreme policies, so $B(s)$ contains it and the metapolicy over the d extreme policies is also optimal.

Hence, the algorithm of [12] can be used unchanged with only the d extreme policies, and each iteration costs $|\mathcal{U}||\mathcal{E}|d$ instead of $|\mathcal{U}||\mathcal{E}||\Pi_{CCS}|$. Note that this requires the extreme policies to be optimal from every exit, so their initial state distribution ν_0 must cover the exits, as [12] assume with $\mathcal{E} \subset \text{supp}(\nu_0)$.

This does not hold for stochastic transitions because a policy can then split its probability between exits, and its return becomes a weighted sum over them, $\sum_k w_k \mathbb{E}[\gamma^T \mathbf{1}\{\text{exit } k\}]$, with T the exit time, which can exceed $\max_k w_k \psi_k^{\pi_k}(s,a)$. For example, in the double slit domain (Figure 3a), the optimum keeps several exits open.

4.2 SF-FSA-VI for stochastic domains

In such environments, the extreme policies are not sufficient to complete every task optimally, since the optimal Q-function is no longer attained by a single exit. Hence, zero-shot optimality requires the whole CCS. On the other hand, GPI can achieve the optimal policy with far fewer policies than the whole CCS. However, the backward propagation of SF-FSA-VI makes us compute the whole CCS, because it needs not only the optimal policy for each weight, but also its value at the exits. Given this, we propose Lazy SF-FSA-VI (Algorithm 4), which only requires this value at the weights that the task visits.

The algorithm starts from the d extreme policies, runs Eq. (15) until convergence, and checks whether GPI is exact at every $\mathbf{w}(u)$. Let $V^{GPI}(s) = \max_a Q^{GPI}_{\mathbf{w}}(s,a)$. Since $V^{\pi_j} \leq V^{GPI}$, every j satisfies $Q^{\pi_j}_{\mathbf{w}} = Q_{V^{\pi_j}} \leq Q_{V^{GPI}}$, hence $Q^{GPI}_{\mathbf{w}} \leq Q_{V^{GPI}}$ and also $Q^{GPI}_{\mathbf{w}} \leq Q^*_{\mathbf{w}}$. Therefore, the instant GPI value is optimal if and only if $Q^{GPI}_{\mathbf{w}} = Q_{V^{GPI}}$, which a single backup checks. If the check fails at some $\mathbf{w}(u)$, we call Certify-or-Grow at that weight: the metapolicy over the current policies is added if it is certified, and otherwise the task is solved warm-started with π^{hyb}. The weights are then recomputed, until every automaton state is certified.

Algorithm 4 LAZY SF-FSA-VI(Π)

Require: Π: the d extreme policies, or the policies kept from previous tasks

repeat
  $\{\mathbf{w}(u)\}_{u\in\mathcal{U}} \leftarrow$ fixed point of Eq. (15) over Π
  for every $u \in \mathcal{U}$ such that $Q^{GPI}_{\mathbf{w}(u)} \neq Q_{V^{GPI}}$ **do** ▷ GPI is not exact
    $\pi,\ \boldsymbol{\psi}^{\pi} \leftarrow$ CERTIFY-OR-GROW($\mathbf{w}(u), \Pi$) ▷ Algorithm 3
    $\Pi \leftarrow \Pi \cup \{\pi\}$
  end for
until no policy was added
return $\{\mathbf{w}(u)\}_{u\in\mathcal{U}}$; act with GPI over Π at $\mathbf{w}(u)$ in every u

Proposition 1. *Algorithm 4 terminates, and at termination the weights* $\mathbf{w}(u)$ *are optimal and GPI is optimal in every automaton state. If the low-level MDP is deterministic, it adds no policy.*

Proof. If Π contained an optimal policy for $\mathbf{w}(u)$, then $Q^{GPI}_{\mathbf{w}(u)} = Q^{*}_{\mathbf{w}(u)}$ and the check would pass. Hence, every added policy is new and, since there are finitely many deterministic policies, the algorithm terminates. At termination, $Q^{GPI}_{\mathbf{w}(u)} = Q^{*}_{\mathbf{w}(u)}$ for every u, so Eq. (15) uses the optimal values at the exits, and its fixed point is the optimal one [12]. GPI is then greedy with respect to $Q^{*}_{\mathbf{w}(u)}$ in every automaton state. Finally, in deterministic domains Theorem 2 gives $Q^{GPI}_{\mathbf{w}} = Q^{*}_{\mathbf{w}}$ over the extreme policies for every weight, so the check always passes. □

Every low-level solve happens at a weight that the task needs, and the added policies can be kept for the next tasks, so the set only grows where the tasks require it.

5 Results

We present results on Algorithm 3 for the Deep Sea Treasure and Office domains [19, 11, 12]. As baselines, we take the standard SFOLS algorithm [3] and a warm-started version where the SF table of each new weight is initialized with GPI over all the policies learned so far. We compare them with two versions of CERTIFY-OR-GROW, both warm started. The first one checks the Bellman certificate with the model, while the second assumes a model-free version and completes the table outside $B(s)$ with sampling transitions. All methods learn the SFs with the same tabular SF Q-learning, with $\gamma = 0.99$, $\alpha = 1$ (domain is deterministic) and $\varepsilon = 0.1$, we stop the learning once every

allowed state-action pair doesn't change under a certain tolerance in the SFs, as checking the Q-values is not enough since components with zero weight would never be checked. This stopping rule is conservative and chosen by simplicity, hence, we report both: the steps until the last change (suppossing that an oracle tells when the SFs converged) and all the environment steps used in the conservative rule. In the Office domain the full CCS is too large, so we compute an ε-CCS with $\varepsilon = 0.05$. Results are presented in Figure 1.

All methods recover the same CCS with exact SFs on every seed. However, Certify-or-Grow needs far fewer samples. We compare the steps until convergence (Figure 1b,e), since the total number of steps also depends on the stopping rule, which is not necessarily the same for every method. In the Deep Sea Treasure, the model-free version needs 3.2× fewer steps until convergence than the warm-started SFOLS and 6.5× fewer than the standard SFOLS. In the Office domain, over the 63 queries of the ε-CCS, it needs 2.1× and 20× fewer steps. Moreover, checking the certificate without the model only costs 2–5% more samples than with it. These gains come from the fact that, once the basis is learned, the metapolicy only has to decide which base policy to follow at each state. Since the optimal policy for a new weight is a composition of the base policies over long regions of the state space, this choice remains the same for long stretches and only changes at a few states. Hence, after the basis, a new metapolicy needs on average a few thousand steps to converge in the Deep Sea Treasure, and none in the Office domain, where the GPI warm start is already exact and the only cost left is confirming convergence.

In both domains, composing the extreme policies is enough to reach optimality. In the Office domain the objectives are independent and do not conflict, so even GPI over the 6 corner policies is optimal. The warm start reduces the cost further as the CCS grows, and for the last queries no learning is needed. However, it can be counterproductive at first: in the Office domain, the warm-started SFOLS needs more steps than the standard one in the first queries (Figure 1e), since starting from the values of the previous policies, instead of an optimistic initialization, steers the exploration towards their behaviour.

We repeat the experiment in two stochastic domains (Figure 2): the Office domain where the chosen action is replaced by one of the other three with probability 0.05, and the double slit domain, described below, with a weak wind of ± 1 cells and $\gamma = 0.99$. Under noise a constant step size never settles, hence the SFs are learned with step size $1/(n+1)^{0.7}$, where n is the number of visits of the pair, each learning stops when its greedy policy has not changed for 30 checks of 1000 steps, and the certificate is

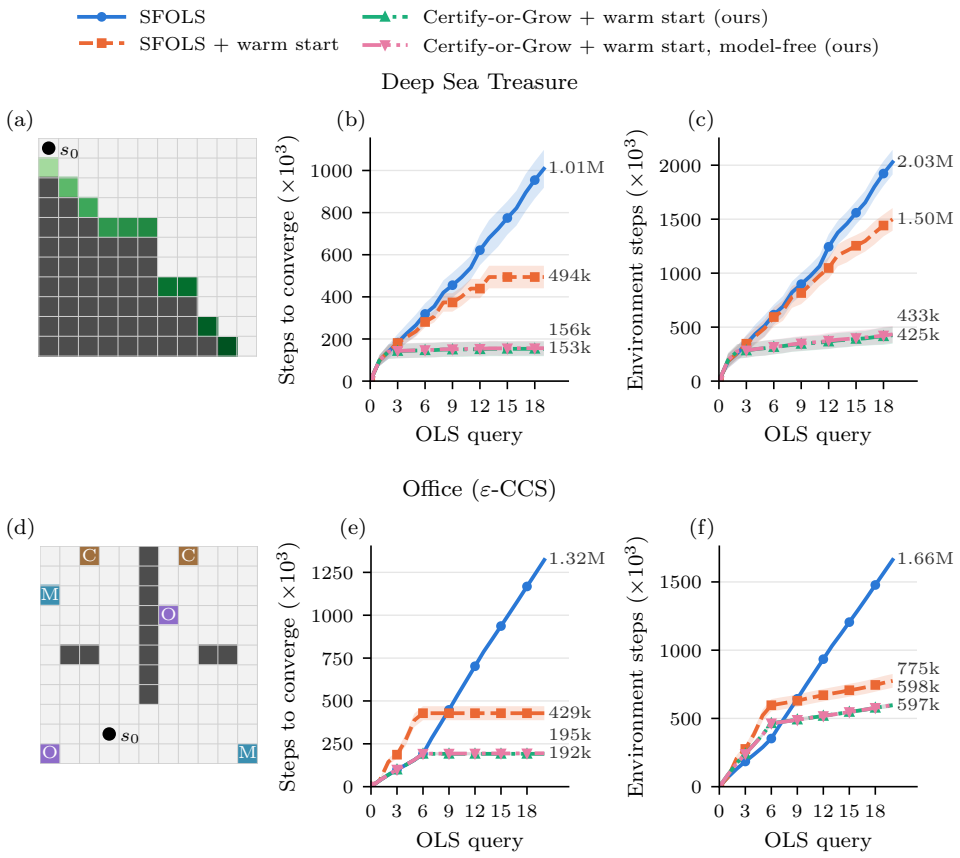


Figure 1: Cumulative sample cost of building the CCS with learned successor features as a function of the OLS query, in the Deep Sea Treasure (top) and Office (bottom). All methods recover the same CCS as with exact successor features. (a) Deep Sea Treasure: the agent starts at s_0 and each treasure ends the episode. (d) Office: walls in dark grey and the six objects (two coffees C, two offices O and two mail M); episodes start at s_0 or at one of the objects. (b, e) Environment steps until convergence. (c, f) All environment steps, including confirming convergence. Mean over 10 seeds, shaded band is ± 1 standard deviation. In Office the variance across seeds is very small, and only the first 20 of the 63 OLS queries of the ε-CCS are shown.

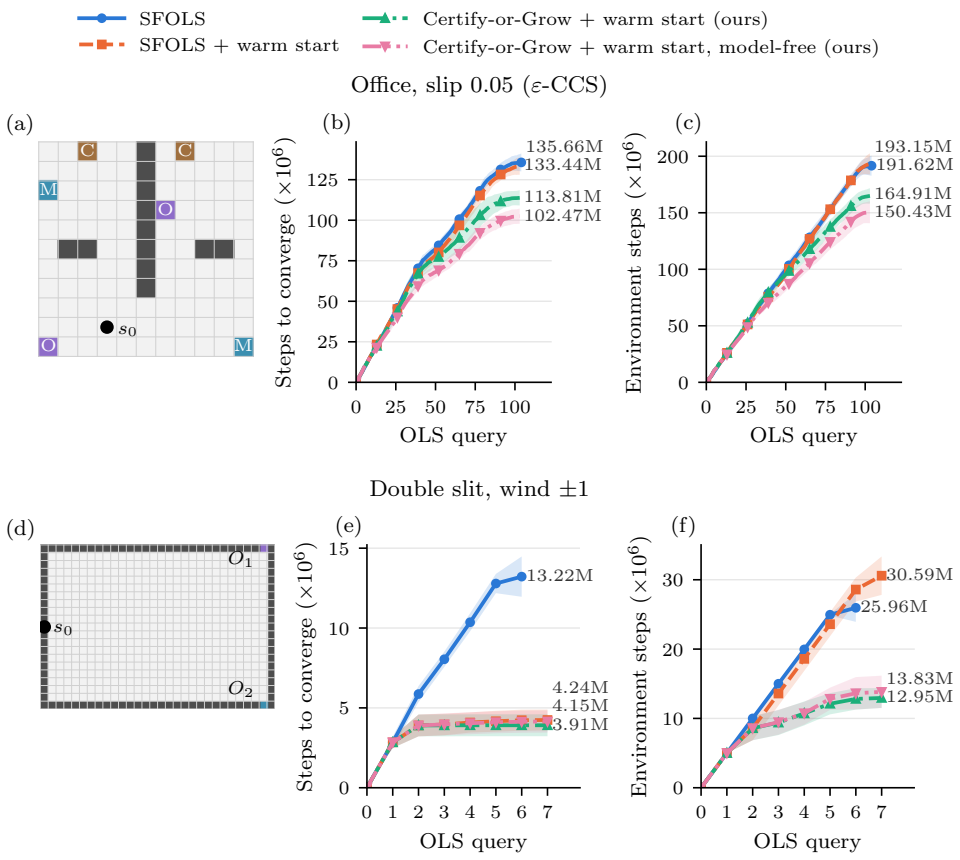


Figure 2: Cumulative sample cost of building the CCS with learned successor features in two stochastic domains, as in Figure 1. (a) Office, where the chosen action is replaced by one of the other three with probability 0.05 (ε-CCS with $\varepsilon = 0.05$). (d) Double slit with wind ± 1 and $\gamma = 0.99$ ($\varepsilon = 10^{-3}$): the agent starts at s_0 and the exits O_1, O_2 end the episode. (b, e) Environment steps until convergence. (c, f) All environment steps. Mean over 10 seeds, shaded band is ± 1 standard deviation.

checked up to a tolerance of 10^{-2}, completing each missing pair with 100 samples in the model-free version. Under noise, two details of Certify-or-Grow matter. First, OLS starts by querying the extreme weights, whose policies are already the basis, so we return them without learning them again. Second, when the certificate fails, the learning over the full action set continues from the table and the visit counts of the metapolicy, as with the warm start of Sec. 3.5, instead of starting from scratch. Without them, Certify-or-Grow needs 10% more steps until convergence than the warm-started SFOLS in the Office domain. In the double slit, Certify-or-Grow needs 3.4× fewer steps until convergence than SFOLS and 8% fewer than the warm-started SFOLS (3.9M against 4.2M), all of them spent on the two extreme policies, since after them every metapolicy is certified and converges immediately. Moreover, it stops much earlier (13M against 31M environment steps): the learning restricted to $B(s)$ settles, while over the full action set the greedy policy keeps switching between nearly tied actions and every learning reaches the cap of 5M steps. In the Office domain the learned metapolicies are noisy, and the certificate with the model fails on 17 of the 98 queries. However, continuing the learning makes these failures cheap, and Certify-or-Grow needs 15% fewer steps until convergence than the warm-started SFOLS (114M against 133M) and 15% fewer environment steps. The model-free certificate is 23% cheaper, but 15% of the metapolicies it certifies are not within the tolerance. Hence, under noise the gain is smaller than in deterministic domains, since most of the cost goes into learning the SFs to the noise level, but composing the basis still saves samples.

On the other hand, the extreme policies are not always enough to compose the optimal policy of every task. For example, consider the double slit domain [12] (Figure 3a). The agent starts at the left of a corridor, always drifts to the right, and a random wind pushes it up or down until it reaches one of the two exits, $\boldsymbol{\phi} = \mathbf{e}_1$ at the top and $\boldsymbol{\phi} = \mathbf{e}_2$ at the bottom. We use a strong wind (± 3 cells per step) and $\gamma = 0.9$. When both exits have a similar weight, the optimal policy keeps both exits open until late, moving forward and letting the wind decide which exit to take. None of the two corner policies has this behaviour, so the optimal value is up to 20.7% higher than the one of the best corner policy, and the CCS has 181 policies.

We compare three ways of acting with the same basis: GPI (Eq. (2)), the metapolicy (Eq. (9)) and π^{hyb} (Eq. (14)) on 200 random held-out weights $\mathbf{w} = (w_1, 1 - w_1)$. We evaluate each policy exactly and count the weights where it is suboptimal (Figure 3b). With only the 2 corner policies, GPI is suboptimal on 58% of the weights and the metapolicy on 54%, whereas π^{hyb} is suboptimal on 15%, and always by less than 0.02% of the optimal

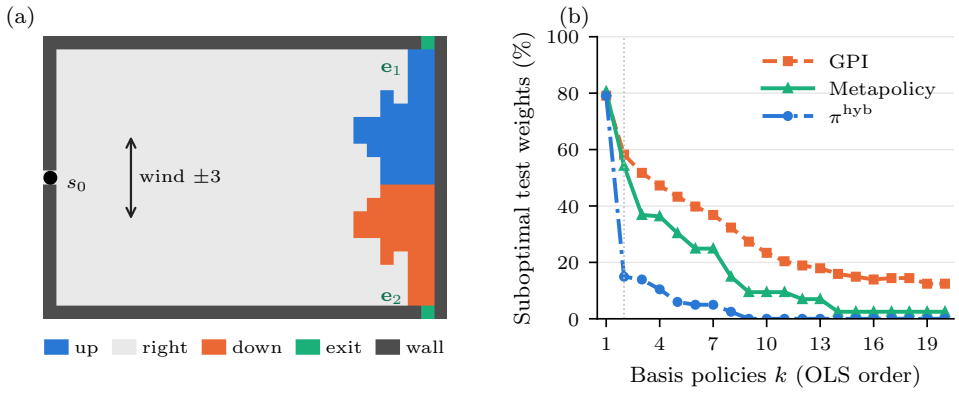


Figure 3: Double slit domain. (a) Action of the optimal policy for $\mathbf{w} = (0.5, 0.5)$ at each cell: it keeps both exits open and only commits to one of them at the end, which neither corner policy does. (b) Percentage of the 200 held-out weights where GPI, the metapolicy and π^{hyb} are suboptimal, with a basis made of the first k policies found by OLS (exact values). The dotted line marks the two corner policies ($k = 2$).

value. The performance of π^{hyb} originates from the possibility of locally taking suboptimal actions while using the metapolicy for future rewards. By contrast, the metapolicy can only switch between the two corner policies at each state. Lastly, GPI evaluates every action following a single base policy afterwards, hence, cannot adapt to a policy that keeps both exits open, see Fig 3a. As we add policies in the order OLS finds them, the metapolicy performance greatly surpasses the GPI one. Furthermore, π^{hyb} becomes optimal on all the held-out weights with 9 policies, while with 20 policies the metapolicy is suboptimal on 2.5% of the weights and GPI on 12.4%. Notice that the number of suboptimal weights can also increase in GPI, for example, from $k = 16$ to $k = 17$ it goes 13.9% to 14.4%. In general, π^{hyb} can replace a large part of the CCS in exchange of performing extra computations for the particular task. In this domain, 9 policies are enough to be optimal on every held-out weight, instead of the 181 required by the full CCS.

Finally, we evaluate Algorithm 4 with exact SFs in two stochastic domains. In the Office domain, the chosen action is replaced by one of the other three with probability $p \in \{0.1, 0.3\}$ ($\gamma = 0.99$), and we use the three tasks of [12] together with 30 random automata. The windy domain ($\gamma = 0.95$) has two stages: the wind of the double slit decides between two exits, A

and B, and from each of them corridors lead to four further exits. 52 of its 60 tasks are of the form "A or B, then a corridor exit", so the first stage must keep both exits open. As in [12], reaching an exit whose symbol has no edge keeps the automaton in the same state, and we discount the transition between automaton states by γ, so that Eq. (15) remains a contraction when an exit can be re-entered immediately. We start either from the 6 extreme policies on every task, or keep the added policies over the sequence of tasks.

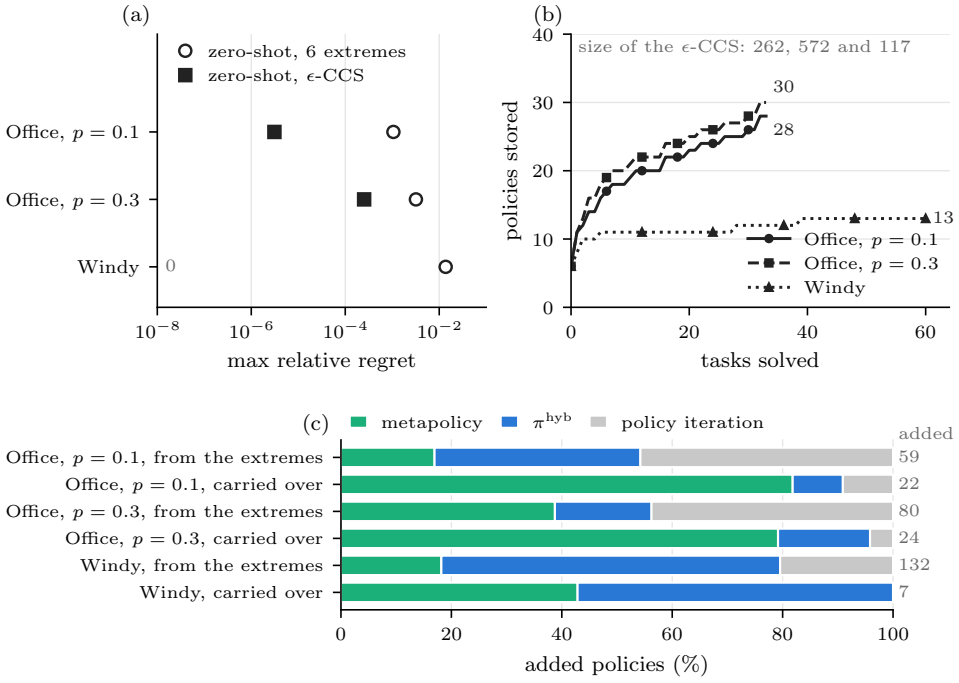


Figure 4: Lazy SF-FSA-VI (Algorithm 4) with exact SFs, starting from the 6 extreme policies. (a) Maximum relative regret over the tasks of zero-shot SF-FSA-VI over the extreme policies and over the whole ε-CCS of SFOLS; Algorithm 4 has zero regret on every task. (b) Policies stored by Algorithm 4 when the added policies are kept over the sequence of tasks. (c) How the added policies were certified: as the metapolicy, as π^{hyb}, or after policy iteration, starting from the extreme policies on every task or keeping the added ones (the number of added policies is on the right).

Zero-shot SF-FSA-VI over the extreme policies is suboptimal by up to 0.11%, 0.32% and 1.4%, and even over the whole ε-CCS of SFOLS (262 and 572 policies, $\varepsilon = 2 \cdot 10^{-3}$) it remains suboptimal in the Office domain

(Figure 4a). In contrast, Algorithm 4 is optimal on all 126 tasks. From the extreme policies, it needs 2 to 2.6 rounds and adds about 2 policies per task. When the added policies are kept, it stores 28, 30 and 13 policies after all the tasks, far fewer than the CCS (Figure 4b). Moreover, most of the added policies are compositions of the stored ones: when they are kept, 91–100% of them are certified metapolicies or π^{hyb}, without any improvement step over the full action set (Figure 4c). In the windy domain, π^{hyb} provides the action that keeps both exits open, as in the double slit. With the model, the cost of Algorithm 4 is comparable to solving the product MDP in the Office domain, and 12× lower in the windy domain, since the stored policies can be reused across tasks. Moreover, a new task becomes zero-shot as the stored set grows: when the added policies are kept, 58% of the Office tasks and 92% of the windy tasks are certified without adding any policy, and 71% and 97% in the second half of the sequence, where a task needs on average 3 certificate checks and 0.03–0.35 new policies. Finally, since each iteration of Eq. (15) costs $|\mathcal{U}||\mathcal{E}||\Pi|$, planning over the at most 30 stored policies is 10–18× cheaper than over the ε-CCS.

Without a model, we follow the comparison of [12] with learned SFs (Figure 5). Every policy, including the extreme ones, is learned by SF Q-learning with step size $1/(n+1)^{0.7}$ from a zero initialization, so that the estimates are biased low and the GPI estimate remains approximately a lower bound. The certificate of Algorithm 4 is replaced by an oracle that checks that, at $\mathbf{w}(u)$, the GPI policy and the GPI estimate are within 10^{-3} and 10^{-2} of the optimum at the start and at the exits. Checking the estimate is necessary, since Eq. (15) reads it at the exits: checking only the policy certified windy tasks whose plan lost 15%. We compare Algorithm 4, growing with Q-learning over all actions or with the metapolicy, with SF-FSA-VI over all the policies learned by SFOLS (up to 100 queries) from the same extreme policies, and with Q-learning on the product MDP. In the windy domain, zero-shot SF-FSA-VI over the learned extreme policies loses 0.9% on average and up to 2.1%, whereas Algorithm 4 brings every task within 0.4% (mean 0.17%) with 0.85 new policies and 1M steps per task, against 127M steps per task for Q-learning on the product MDP. When the added policies are kept, 97% of the tasks need no new policy, so the algorithm is zero-shot again. SFOLS needs 62M steps after the extreme policies for a similar regret, and if its learned policies do not inherit the visit counts of the stored ones, its plan becomes worse as the set grows, since the maximum over many noisy estimates overestimates the values at the exits. In the Office domain the extreme policies are already within 0.33%, and a new task needs 1.5 policies and 0.8M steps (0.14M when the policies are kept), against 2.9M

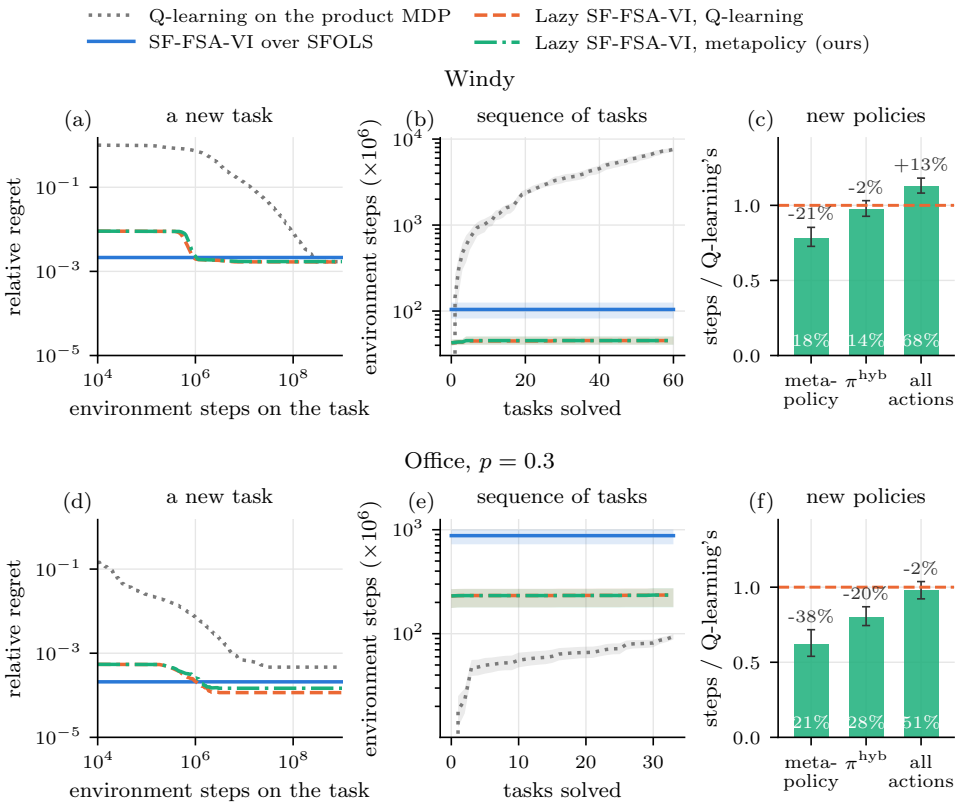


Figure 5: Lazy SF-FSA-VI without a model, in the windy domain (60 tasks, top) and the Office domain with $p = 0.3$ (33 tasks, bottom), mean over 10 seeds. (a, d) A new task: relative regret of the executed plan against the environment steps spent on the task. Lazy SF-FSA-VI starts from the 6 learned extreme policies and grows with the metapolicy or with Q-learning over all actions, Q-learning on the product MDP starts from scratch, and SF-FSA-VI over the policies of SFOLS spends nothing per task (horizontal line at its final regret). (b, e) Cumulative environment steps over the sequence of tasks, including the extreme policies, when the added policies are kept. The band is the range over seeds. (c, f) How the metapolicy growth obtained the new policies from the extreme ones: certified metapolicy, certified π^{hyb}, or learned over all actions, with their share inside the bars, and their environment steps relative to Q-learning at the same weight (dashed line), over 20 seeds with a 95% interval.

for Q-learning on the product MDP and 646M for SFOLS. However, learning the extreme policies accurately at every exit costs 231M steps, so the SFs only pay off after about 85 tasks. Over the four domains (including Office with $p = 0.1$ and the double slit), Lazy SF-FSA-VI needs 2–4× fewer samples than SFOLS in total, and 21–350× fewer after the extreme policies. Finally, the metapolicy growth follows the steps of Algorithm 3 and Sec. 3.5 without a model. The metapolicy is learned on $B(s)$ until its policy has been optimal in $\mathcal{M}_{\mathcal{B}}$ for 10 checks. If it is not certified, its table is completed outside $B(s)$ with 100 samples per pair and π^{hyb} is checked. A certified π^{hyb} is refined on $B(s)$ and its own actions, and otherwise the learning continues over all actions from the same table, so that a wrong decision costs little. Whenever the new policy is a composition, the growth is cheaper than Q-learning at the same weight (Figure 5c, f): a certified metapolicy needs 21% and 38% fewer steps in the windy domain and in the Office domain with $p = 0.3$, and a certified π^{hyb} 2% and 20% fewer. These are 32% of the new policies in the windy domain and 49% in the Office domain with $p = 0.3$, whereas the remaining ones pay the metapolicy learning before the fallback (13% more steps in the windy domain). Hence, per task both growths cost about the same (0.97–1.03×), and the metapolicy only saves samples on the policies that are compositions of the stored ones.

6 Conclusion

We have studied how to speed up the construction of the convex coverage set in four environments, both deterministic and stochastic, and presented a new method that computes the CCS faster than the standard SFOLS algorithm, together with a proof of its convergence. Our method combines the optimal actions of the policies in the basis into a metapolicy that switches between them across the state space. Results show that it speeds up the construction of the CCS and finds the optimal policies with fewer samples in deterministic domains, while under stochastic transitions the gain is smaller, since learning the successor features of the basis dominates the cost. Moreover, our method identifies the base policies of an MDP, which, combined with GPI, can be used to solve new tasks without computing the full CCS. For non-Markovian tasks, the extreme policies are enough for SF-FSA-VI in deterministic domains, and in stochastic domains Lazy SF-FSA-VI only adds policies at the weights that a task needs, so that most new tasks become zero-shot once a few policies are stored. Without a model, its main limitation is the certificate, which we checked with an oracle: a sound model-free

certificate, for example with confidence bounds from the visit counts of the learned successor features, is left for future work. Our results apply to both deterministic and stochastic MDPs, and could be extended to continuous MDPs. In the future, we would like to test the algorithm on more challenging domains, where the rewards and the model have to be learned jointly with the successor features, and to study its behaviour when the successor features are learned with function approximation. Such experiments will establish real benchmarks for practical applications.

A Convergence of the metapolicy recursion

We prove the convergence of Algorithm 2, as it is implemented, and then show how to make $\widehat{\boldsymbol{\psi}}$ converge to the SFs of a single policy.

Theorem 3. *Assume that every pair (s, a) is updated infinitely often and that the step sizes satisfy the Robbins–Monro conditions [24]. Then the policy returned by Algorithm 2 is optimal in $\mathcal{M}_{\mathcal{B}}$ after a finite number of updates. Moreover, if the best action of $\mathcal{M}_{\mathcal{B}}$ is unique at every state, then $\widehat{\boldsymbol{\psi}}_t(s, a)$ converges to $\boldsymbol{\psi}^{\pi_{\text{meta}}}(s, a)$ for every (s, a).*

Proof. Let $Q_t(s, a) = \widehat{\boldsymbol{\psi}}_t(s, a) \cdot \mathbf{w}$ be the scalarized table at update t, $\hat{\jmath}_t(s)$ the policy maximizer selected at state s, and $J^*(s) = \{j : Q_{V_{\text{meta}}}(s, \pi_j(s)) = V_{\text{meta}}(s)\}$ the base policies whose action is optimal in $\mathcal{M}_{\mathcal{B}}$ at state s. To show that the returned policy is optimal, we need to show that it exists T such that $\hat{\jmath}_t(s) \in J^*(s)$ for $t \geq T$.

The update of $\widehat{\boldsymbol{\psi}}$ is a vector operator that is not a contraction in general, since two different tables may bootstrap from different maximizers $\hat{\jmath}$. However, since $\hat{\jmath}_t$ maximizes Q_t, taking the dot product of the update with $\mathbf{w}$ gives

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha_t(s, a)\Big[\boldsymbol{\phi}(s, a, s') \cdot \mathbf{w} + \gamma \max_j Q_t\big(s', \pi_j(s')\big) - Q_t(s, a)\Big],$$

which is Q-learning on $\mathcal{M}_{\mathcal{B}}$ [25], extended to the pairs with $a \notin B(s)$, whose entries are updated but never used in the bootstrap. Hence, under the Robbins–Monro conditions and with every pair updated infinitely often, $Q_t \to Q_{V_{\text{meta}}}$ almost surely [26].

Now let $\delta = \min_s \min_{j \notin J^*(s)} \big(V_{\text{meta}}(s) - Q_{V_{\text{meta}}}(s, \pi_j(s))\big) > 0$ be the gap between the optimal and the best non-optimal action of $\mathcal{M}_{\mathcal{B}}$. Then there exists T such that $\|Q_t - Q_{V_{\text{meta}}}\|_\infty < \delta/2$ for every $t \geq T$, which implies that for $j \notin J^*(s)$ and $j^* \in J^*(s)$ we then have

$$Q_t(s, \pi_j(s)) < Q_t(s, \pi_{j^*}(s)),$$

so the maximizer $\hat{\jmath}_t(s)$ is in $J^*(s)$, and the greedy policy is optimal in $\mathcal{M}_{\mathcal{B}}$.

Finally, if the best action is unique at every state, the bootstrap always follows π_{meta} for $t \geq T$, and the update becomes

$$\widehat{\boldsymbol{\psi}}_{t+1}(s,a) = \widehat{\boldsymbol{\psi}}_t(s,a) + \alpha_t(s,a)\big[\boldsymbol{\phi}(s,a,s') + \gamma\, \widehat{\boldsymbol{\psi}}_t\big(s', \pi_{\text{meta}}(s')\big) - \widehat{\boldsymbol{\psi}}_t(s,a)\big],$$

which is the TD update of the SFs of the fixed policy π_{meta}. For the same reasons as Q_t, it converges to $\boldsymbol{\psi}^{\pi_{\text{meta}}}(s,a)$ almost surely. □

Theorem 3 is enough for the Bellman certificate, which only uses the scalar values $\widehat{\boldsymbol{\psi}} \cdot \mathbf{w}$. However, $\widehat{\boldsymbol{\psi}}$ itself does not necessarily converge. When two actions are tied, their order can change at every update, and if their SFs differ, $\widehat{\boldsymbol{\psi}}$ is pulled towards a different target at each update, while its scalarization still converges. If the SFs of a single metapolicy are needed, for example, to add it to the CCS or to use it with GPI, the ties can be broken easily with a small tolerance and a fixed order over the basis.

References

[1] D. M. Roijers et al. "A Survey of Multi-Objective Sequential Decision-Making". In: *Journal of Artificial Intelligence Research* 48 (2013), pp. 67–113.

[2] Conor F. Hayes et al. "A Practical Guide to Multi-Objective Reinforcement Learning and Planning". In: *Autonomous Agents and Multi-Agent Systems* 36 (2022), p. 26. arXiv: 2103.09568.

[3] Lucas N. Alegre, Ana L. C. Bazzan, and Bruno C. da Silva. *Optimistic Linear Support and Successor Features as a Basis for Optimal Policy Transfer*. 2022. arXiv: 2206.11326.

[4] D. M. Roijers. "Multi-objective decision-theoretic planning". In: (2016).

[5] André Barreto et al. "Fast reinforcement learning with generalized policy updates". In: *Proceedings of the National Academy of Sciences of the United States of America* 117.48 (2020), pp. 30079–30087.

[6] Tom Zahavy et al. *Discovering a set of policies for the worst case reward*. 2021. arXiv: 2102.04323.

[7] Safa Alver and Doina Precup. "Constructing a Good Behavior Basis for Transfer using Generalized Policy Updates". In: *International Conference on Learning Representations*. 2022.

[8] Lucas N. Alegre et al. *Constructing an Optimal Behavior Basis for the Option Keyboard*. 2025. arXiv: 2505.00787.

[9] Wilka Carvalho et al. *Combining Behaviors with the Successor Features Keyboard*. 2023. arXiv: 2310.15940.

[10] Dimitris Michailidis et al. "Scalable Multi-Objective Reinforcement Learning with Fairness Guarantees using Lorenz Dominance". In: *Journal of Artificial Intelligence Research* 85 (2026).

[11] Florian Felten et al. "A Toolkit for Reliable Benchmarking and Research in Multi-Objective Reinforcement Learning". In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 23671–23700.

[12] David Kuric et al. "Planning with a Learned Policy Basis to Optimally Solve Complex Tasks". In: *Proceedings of the International Conference on Automated Planning and Scheduling* 34 (2024), pp. 333–341.

[13] Richard S. Sutton, Doina Precup, and Satinder Singh. "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning". In: *Artificial Intelligence* 112.1 (1999), pp. 181–211.

[14] Andrew Li et al. "Reward Machines for Deep RL in Noisy and Uncertain Environments". In: *Advances in Neural Information Processing Systems*. Vol. 37. Curran Associates, Inc., 2024, pp. 110341–110368.

[15] Kristof Van Moffaert and Ann Nowé. "Multi-Objective Reinforcement Learning using Sets of Pareto Dominating Policies". In: *Journal of Machine Learning Research* 15.107 (2014), pp. 3663–3692.

[16] André Barreto et al. "Successor features for transfer in reinforcement learning". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. Curran Associates Inc., 2017, pp. 4058–4068.

[17] Jonathan Hunt et al. "Composing Entropic Policies using Divergence Correction". In: *Proceedings of the 36th International Conference on Machine Learning*. PMLR, 2019, pp. 2911–2920.

[18] Lucas N. Alegre et al. "Sample-Efficient Multi-Objective Learning via Generalized Policy Improvement Prioritization". In: *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 2023, pp. 2003–2012.

[19] Amber Cassimon et al. *A Review of the Deep Sea Treasure problem as a Multi-Objective Reinforcement Learning Benchmark*. 2024. arXiv: 2110.06742.

[20] Mark Nemecek and Ronald Parr. "Policy Caches with Successor Features". In: *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 2021, pp. 8025–8033.

[21] Siddarth Chandrasekar and Marlos C. Machado. "Towards An Option Basis To Optimize All Rewards". In: *Workshop on Reinforcement Learning Beyond Rewards @ Reinforcement Learning Conference 2025*. 2025.

[22] Diana Borsa et al. *Universal Successor Features Approximators*. 2018. arXiv: 1812.07626.

[23] André Barreto et al. *The Option Keyboard: Combining Skills in Reinforcement Learning*. 2021. arXiv: 2106.13105.

[24] Herbert Robbins and Sutton Monro. "A Stochastic Approximation Method". In: *The Annals of Mathematical Statistics* 22.3 (1951), pp. 400–407.

[25] Christopher J. C. H. Watkins and Peter Dayan. "Q-learning". In: *Machine Learning* 8.3 (1992), pp. 279–292.

[26] John N. Tsitsiklis. "Asynchronous Stochastic Approximation and Q-Learning". In: *Machine Learning* 16.3 (1994), pp. 185–202.